

Bitwise operators look at:

```
10 = 1010  
5  = 0101
```



Then they operate **bit by bit**.

Why are they called “Bitwise”?

Because:

```
bit = smallest unit (0 or 1)  
wise = operation done bit-by-bit
```



So:

```
Bitwise = operate on each binary digit
```



First: Binary Basics (VERY IMPORTANT)

Let's convert simple numbers:

Decimal	Binary
5	0101
6	0110
10	1010

You don't need to master binary now — just understand it exists.



1. AND Operator (&)

Rule:

```
1 & 1 = 1  
anything else = 0
```



Example:

```
5 = 0101  
3 = 0011
```



Now AND:

```
  0101  
& 0011  
-----  
  0001
```



Result:

```
1
```



Code:

```
</> C++  
  
int a = 5;  
int b = 3;  
  
cout << (a & b);
```



Output:

```
1
```



2. OR Operator (|)

Rule:

```
1 | 1 = 1
1 | 0 = 1
0 | 0 = 0
```



Example:

```
5 = 0101
3 = 0011
```



OR:

```
  0101
| 0011
-----
  0111
```



Result:

```
7
```



Code:

```
<> C++
cout << (5 | 3);
```



Output:

```
7
```



3. XOR Operator (^)

Rule:

```
same bits → 0  
different bits → 1
```



Example:

```
5 = 0101  
3 = 0011
```



XOR:

```
  0101  
^ 0011  
-----  
  0110
```



Result:

```
6
```



Code:

```
<> C++  
  
cout << (5 ^ 3);
```



Output:

```
6
```



4. NOT Operator (~)

Rule:

```
flips bits:  
1 → 0
```



0 → 1

Example:

5 = 00000101



NOT:

~5 = 11111010



This becomes a **negative number in C++ (two's complement system)**.

So output is usually:

-6



Code:

```
</> C++  
  
cout << ~5;
```



Output:

-6



5. LEFT SHIFT (<<)

This shifts bits to the left (multiply by 2)

Example:

5 = 00000101



Shift left by 1:

00001010 = 10



So:

5 << 1 = 10



Code:

```
</> C++  
  
cout << (5 << 1);
```



Output:

10



6. RIGHT SHIFT (>>)

This shifts bits to the right (divide by 2)

Example:

5 = 00000101



Shift right:

00000010 = 2



So:

5 >> 1 = 2



Code:

```
</> C++
```

```
cout << (5 >> 1);
```



Output:

```
2
```



SUPER SIMPLE SUMMARY

Operator	Name	Meaning
&	AND	both must be 1
	OR	OR
^	XOR	different = 1
~	NOT	flips bits
<<	left shift	multiply by 2
>>	right shift	divide by 2



EASY WAY TO REMEMBER

Think like switches 💡:

```
AND → both must be ON  
OR → any ON works  
XOR → only one ON  
NOT → flip switch  
SHIFT → move bits left/right
```



REAL USES (Important)

Bitwise operators are used in:

- game development 🎮
- system programming 💻
- compression
- encryption 🔒
- performance optimization

If you want next step, I can show you:

- 👉 "Bitwise operators exam questions (step-by-step solving)"
- 👉 "How binary conversion actually works (super easy method)"
- 👉 "Why XOR is used in real programming tricks"

Just tell me 👍



assignment operators

Assignment Operators in C++

Assignment operators are used to **assign values to variables**. They can also **combine assignment with another operation** (like addition or multiplication). Let's go step-by-step.



1 Simple Assignment (=)

This is the most basic assignment operator.

Meaning:

Assign the value on the right to the variable on the left.

Example:


```
</> C++
```



```
int a;  
a = 10;
```

Now:

```
a = 10
```



Another example:

```
</> C++
```



```
int x = 5;  
int y = x;
```

Now:

```
y = 5
```



2 Addition Assignment (+=)

Adds a value to a variable and assigns the result back.

Syntax

```
</> C++
```



```
a += b;
```

Equivalent to

```
</> C++
```



```
a = a + b;
```

Example

```
</> C++
```



```
int a = 5;
```

```
a += 3;
```

Result:

```
a = 8
```



3 Subtraction Assignment (-=)

Subtracts and assigns the result.

Syntax

```
</> C++
```

```
a -= b;
```



Equivalent to

```
</> C++
```

```
a = a - b;
```



Example

```
</> C++
```

```
int a = 10;  
a -= 4;
```



Result:

```
a = 6
```



4 Multiplication Assignment (*=)

Multiplies and assigns the result.

Syntax

```
</> C++
```



```
a *= b;
```

Equivalent to

```
</> C++
```



```
a = a * b;
```

Example

```
</> C++
```



```
int a = 6;  
a *= 2;
```

Result:

```
a = 12
```



5 Division Assignment (/=)

Divides and assigns the result.

Syntax

```
</> C++
```



```
a /= b;
```

Equivalent to

```
</> C++
```



```
a = a / b;
```

Example

```
</> C++
```



```
int a = 20;  
a /= 4;
```

Result:

```
a = 5
```



6 Modulus Assignment (%=)

Finds the remainder and assigns it.

Syntax

```
</> C++  
a %= b;
```



Equivalent to

```
</> C++  
a = a % b;
```



Example

```
</> C++  
int a = 10;  
a %= 3;
```



Result:

```
a = 1
```



Quick Summary Table

Operator	Meaning	Equivalent
=	Assign	a = b

Operator	Meaning	Equivalent
<code>+=</code>	Add and assign	<code>a = a + b</code>
<code>-=</code>	Subtract and assign	<code>a = a - b</code>
<code>*=</code>	Multiply and assign	<code>a = a * b</code>
<code>/=</code>	Divide and assign	<code>a = a / b</code>
<code>%=</code>	Modulus and assign	<code>a = a % b</code>

Example Program

 C++



```
#include <iostream>
using namespace std;

int main()
{
    int a = 10;

    a += 5; // 15
    a -= 3; // 12
    a *= 2; // 24
    a /= 4; // 6
    a %= 5; // 1

    cout << "Final value: " << a;

    return 0;
}
```

Output

Final value: 1



 Simple idea to remember:

```

+=  add and store
-=  subtract and store
*=  multiply and store
/=  divide and store
%=  remainder and store

```



If you want, I can also [↪ explain Arithmetic Operators, Relational Operators, Logical Operators, and Bitwise Operators in one easy comparison chart](#) (very useful for exams).



<<=, >>=, &=

These are **compound assignment bitwise operators** in C++. They combine a **bitwise operation + assignment** in one step. 🖥️

◆ 1. Left Shift Assignment (<<=)

Meaning:

```

</> C++
a <<= b;

```



is the same as

```

</> C++
a = a << b;

```



It shifts the bits of **a** to the left by **b** positions and stores the result in **a**.

Example

```

</> C++

```



```
#include <iostream>
using namespace std;

int main()
{
    int a = 5;    // binary: 0101
    a <<= 1;

    cout << a;
}
```

Step

```
5   = 00000101
<<1
-----
    00001010 = 10
```



✓ Output

```
10
```



💡 Left shift usually multiplies by 2.

◆ 2. Right Shift Assignment (>>=)

Meaning

```
</> C++
a >>= b;
```



is the same as

```
</> C++
a = a >> b;
```



It shifts bits to the right.

Example

</> C++



```
#include <iostream>
using namespace std;

int main()
{
    int a = 8;    // 1000
    a >>= 1;

    cout << a;
}
```

Step

```
8  = 00001000
>>1
-----
00000100 = 4
```



✓ Output

4



💡 Right shift usually **divides by 2**.

◆ 3. Bitwise AND Assignment (&=)

Meaning

</> C++



```
a &= b;
```

is the same as

</> C++



```
a = a & b;
```

Example

</> C++

```
#include <iostream>
using namespace std;

int main()
{
    int a = 6;    // 0110
    int b = 3;    // 0011

    a &= b;

    cout << a;
}
```

Step

0110
& 0011

0010 = 2

✔ Output

2

 Quick Summary

Operator	Meaning	Equivalent
<<=	Left shift and assign	a = a << b
>>=	Right shift and assign	a = a >> b
&=	Bitwise AND and assign	a = a & b

 Simple Memory Trick

```
<<=  shift left and store
>>=  shift right and store
&=   AND and store
```



✅ If you'd like, I can also ↪ show the remaining bitwise assignment operators that usually come in exams:

```
|=  (OR assignment)
^=  (XOR assignment)
```



and ↪ a single program that demonstrates all of them together.

📄 👍 💬 ↺ ...

c=5; c

It looks like you wrote:

```
</> C++

c = 5;
c
```



Let's explain what this means in C/C++. 🙋

1 c = 5;

This is an **assignment statement**.

It assigns the value 5 to the variable c.

Example:

```
</> C++

int c;
```



```
c = 5;
```

Now the value stored in `c` is:

```
c = 5
```

**2**`c`

If you only write:

```
</> C++
```



```
c
```

by itself in a program, **nothing happens**.

The program just references the variable but **does not print or change it**.

To **see the value**, you must print it.

Example:

```
</> C++
```



```
#include <iostream>
using namespace std;

int main()
{
    int c;
    c = 5;

    cout << c;

    return 0;
}
```

**Output**

```
5
```



3 Important Note

In programming:

- `=` → assignment operator
- `c = 5` → store value 5 in c

✓ Summary

Statement	Meaning
<code>c = 5;</code>	Assign 5 to variable <code>c</code>
<code>c</code>	Refers to the variable (does nothing alone)

If you want, I can also ↪ explain a tricky exam question about `c = 5` , `c += 5` , `c <= 1` , etc. that teachers often give. 💡



`c = 15; c >>= 2;` to find c you have to convert 15 in binary form?

Yes 👍 To evaluate `c >>= 2` , it helps to **convert the number to binary**, because the **right-shift operator moves bits to the right**.

Let's solve it step by step.

Step 1 Initial value

```
</> C++
```



```
c = 15;
```

Binary of 15:

```
15 = 00001111
```



Step 2 Apply Right Shift

```
</> C++
```



```
c >>= 2;
```

This means:

```
c = c >> 2
```



Shift the bits 2 places to the right.

```
Before shift: 00001111
```



```
After >>1 : 00000111
```

```
After >>2 : 00000011
```

Step 3 Convert back to decimal

```
00000011 = 3
```



✅ Final value

```
c = 3
```



💡 Shortcut Rule

For positive numbers:

```
Right shift by n ≈ divide by 2n
```



So:

```
15 >> 2 = 15 / 4 = 3
```



(integer division)

✓ Final Answer

`c = 3`



✓ Quick Tip for Exams

Expression	Result
<code>8 >> 1</code>	4
<code>16 >> 2</code>	4
<code>15 >> 2</code>	3

If you want, I can also show why `15 >> 2` becomes 3 visually in binary (very easy trick teachers use in exams).



`c &= 2` this one we need to convert both the value of `c` and `2` to binary right?

Yes 👍 **Correct.** For the operator `&=` you should convert **both numbers to binary**, because it uses the **bitwise AND operation**.

Let's understand it step by step.

1 Expression

</> C++



```
c &= 2;
```

This means:

</> C++



```
c = c & 2;
```

So the value of c is ANDed with 2, and the result is stored back in c.

2 Example (Assume `c = 15`)

</> C++



```
c = 15;
```

```
c &= 2;
```

3 Convert both numbers to binary

Decimal	Binary
15	1111
2	0010

4 Apply Bitwise AND (&)

Rule of AND:

A	B	A & B
0	0	0
0	1	0
1	0	0

A	B	A & B
1	1	1

Now apply it:

```
  1111
& 0010
-----
  0010
```



5 Convert result to decimal

```
0010 = 2
```



✓ Final value

```
c = 2
```



✓ Final Answer

If

```
</> C++
```



```
c = 15;
c &= 2;
```

Then

```
c = 2
```



💡 Important Tip

For bitwise operators (`&`, `|`, `^`):

- Convert numbers to **binary**
- Apply the operation **bit by bit**

If you want, I can also ↪ show the easiest exam trick to solve `&`, `|`, `^`, `<<`, `>>` questions in 5 seconds without writing full binary.



miscellaneous operators

In C/C++, **Miscellaneous Operators** are special operators that don't fit into the main categories (arithmetic, logical, relational, etc.). They perform **specific tasks like checking size, selecting values, or accessing members.** 🧠💻

1 `sizeof` Operator

The `sizeof` operator is used to find the **size of a variable or data type in bytes.**

Example:

```
</> C++  
  
#include <iostream>  
using namespace std;  
  
int main() {  
    int a;  
    cout << sizeof(a);  
}
```

✅ If `int` takes **4 bytes**, the output will be:

4

You can also use it with data types:

</> C++



```
sizeof(int)
sizeof(float)
sizeof(char)
```

2 Conditional (Ternary) Operator `?:`

This operator is a short form of an if-else statement.

Syntax

</> C++



```
condition ? expression1 : expression2;
```

- If the **condition** is true → `expression1` runs
- If the **condition** is false → `expression2` runs

Example

</> C++



```
int a = 10, b = 5;

int max = (a > b) ? a : b;
cout << max;
```

✓ Output

10



3 Comma Operator `,`

The **comma operator** allows multiple expressions to be evaluated in one statement.
The **last value** is returned.

Example

</> C++



```
int a, b;  
a = (b = 3, b + 2);
```

Steps:

```
b = 3  
b + 2 = 5
```



So:

```
a = 5
```



4 Pointer Operator *

The * operator is used to **declare a pointer**.

Example

</> C++



```
int a = 10;  
int *p;  
  
p = &a;
```

Here:

- p is a **pointer**
- It stores the **address of a**

5 Address-of Operator &

The & operator gives the **memory address of a variable**.

Example

</> C++



```
int a = 10;
cout << &a;
```

Output will be the **memory address** of `a` .

 Quick Summary

Operator	Name	Purpose
<code>sizeof</code>	Size operator	Finds memory size
<code>?:</code>	Conditional operator	Short form of if–else
<code>,</code>	Comma operator	Multiple expressions
<code>*</code>	Pointer operator	Declares pointer
<code>&</code>	Address operator	Gets memory address

 Exam Tip

Most textbooks list these 3 as the main miscellaneous operators:

```
sizeof
?: (conditional operator)
, (comma operator)
```



If you want, I can also ↪ explain all C++ operators in one full chart (8 types) that usually appears in programming exams. 📄



.,arrow and cast

These are also **Miscellaneous Operators in C/C++**. They are used for **accessing members and type conversion**. Let's explain them clearly. 📌

1 Dot Operator (.)

The dot operator is used to access members of a structure or object.

Example (Structure)

```
</> C++  
  
#include <iostream>  
using namespace std;  
  
struct Student  
{  
    int age;  
};  
  
int main()  
{  
    Student s1;  
    s1.age = 20;  
  
    cout << s1.age;  
}
```

✅ Explanation

- `s1.age` → access the **age member** of structure `s1`.

2 Arrow Operator (->)

The arrow operator is used to access structure members using a pointer.

Example

```
</> C++  
  
#include <iostream>  
using namespace std;  
  
struct Student
```

```
{
    int age;
};

int main()
{
    Student s1;
    Student *p;

    p = &s1;
    p->age = 20;

    cout << p->age;
}
```

✓ Explanation

- `p->age` means

`(*p).age`



So it accesses the member through a pointer.

3 Cast Operator (Type Casting)

The cast operator is used to convert one data type into another.

Syntax

`<> C++`

`(type) value`



Example

`<> C++`

```
#include <iostream>
using namespace std;

int main()
{
    int a = 5, b = 2;
```



```
float c = (float)a / b;

cout << c;
}
```

Result

2.5



Without casting:

5 / 2 = 2



With casting:

5 / 2 = 2.5



Quick Summary

Operator	Name	Use
.	Dot operator	Access member of structure/object
->	Arrow operator	Access member through pointer
(type)	Cast operator	Convert one data type to another



Easy way to remember

. → normal object member access
-> → pointer member access
(type) → change data type



If you'd like, I can also ↪ show a very common exam question that compares . and -> with a diagram, which makes it **super easy to understand**.



comma operator

The **Comma Operator (,)** in C/C++ allows multiple expressions to be written in a single statement. The expressions are **evaluated from left to right**, and the **value of the last expression becomes the result**. 🖥️

◆ Basic Idea

Syntax:

</> C++



```
expression1, expression2, expression3;
```

- All expressions are **executed in order**
- The **last expression gives the final value**

◆ Example 1

</> C++



```
#include <iostream>
using namespace std;

int main()
{
    int a;

    a = (3, 5);

    cout << a;

    return 0;
}
```


Step-by-step

(3,5)



- 3 is evaluated first
- 5 is evaluated second
- The **last value (5)** is assigned to a

✓ Output

5



◆ Example 2

</> C++



```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    a = (b = 3, b + 2);

    cout << a;

    return 0;
}
```

Steps

b = 3
b + 2 = 5



So:

a = 5



✓ Output



◆ Example 3 (Common in loops)

The comma operator is often used in **for loops**.

</> C++

```
for(int i = 0, j = 5; i < j; i++, j--)  
{  
    cout << i << " " << j << endl;  
}
```



Here:

```
i++, j--
```



Both expressions run in **one statement**.

Key Points

Feature	Description
Symbol	
Evaluation order	Left → Right
Result	Value of the last expression

Simple Memory Trick

Comma operator → Do many things,
Return the last thing.



✅ If you want, I can also show you ↪ a tricky exam question using the comma operator that confuses many students (but becomes easy once you know the trick).



```
#include <iostream>
using namespace std;

int main()
{
    int i, j;
    j = 10;
    i = (j++);
    cout << i;

    return 0;
}
```

Let's go through your code **step by step** to understand what happens. 🖥

</> C++



```
#include <iostream>
using namespace std;

int main()
{
    int i, j;
    j = 10;
    i = (j++);
    cout << i;

    return 0;
}
```

1 Variable Declaration

```
</> C++
```



```
int i, j;
```

Two integer variables are created:

```
i → empty
```

```
j → empty
```



2 Assign Value to j

```
</> C++
```



```
j = 10;
```

Now:

```
j = 10
```



3 Post Increment

```
</> C++
```



```
i = (j++);
```

`j++` is a **post-increment operator**.

Meaning:

```
Use the current value first  
Then increase it by 1
```



Step-by-step

1. Current value of `j` = 10
2. That value is assigned to `i`
3. Then `j` increases by 1

So:

```
i = 10  
j = 11
```



4 Output

```
</> C++  
  
cout << i;
```



The program prints:

```
10
```



Final Values

Variable	Value
----------	-------

i	10
---	----

j	11
---	----

Output:

```
10
```



Important Concept

There are two increment types:

Post Increment

```
j++
```



Use first, then increase.

Example:

i = j++

Pre Increment

++j

Increase first, then use.

Example:

i = ++j

⚡ Quick Comparison

Expression	i	j
i = j++	10	11
i = ++j	11	11

If you want, I can also show you ↪ a small trick professors use in exams like `i = j++ + ++j` that confuses many students.

```
#include <iostream>
using namespace std;

int main()
{
    int i, j;
```

```
j = 10;  
i = (j++, j + 100, 999 + j);  
cout << i;  
  
return 0;  
}
```

This example uses the **comma operator**, so the expressions inside the parentheses are **evaluated from left to right**, but **only the value of the last expression is assigned to `i`**.

Let's break it down. 📌

</> C++



```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int i, j;  
    j = 10;  
    i = (j++, j + 100, 999 + j);  
    cout << i;  
  
    return 0;  
}
```

1 Initial Value

</> C++



```
j = 10;
```

So:

```
j = 10
```



2 Comma Operator Expression

```
</> C++
```



```
i = (j++, j + 100, 999 + j);
```

There are **three expressions** separated by commas.

First expression

```
j++
```



- Post-increment
- Use current value, then increase

So:

```
j becomes 11
```



Second expression

```
j + 100
```



Now `j = 11`.

```
11 + 100 = 111
```



This value is **calculated but discarded** (because it is not the last expression).

Third expression (Final Value)

```
999 + j
```



```
j = 11
```

```
999 + 11 = 1010
```



The last expression becomes the value of the whole comma expression.

So:


```
i = 1010
```



3 Output

```
<> C++
```

```
cout << i;
```



Output:

```
1010
```



Final Values

Variable	Value
j	11
i	1010

Program Output:

```
1010
```



Key Rule of the Comma Operator

```
(expression1, expression2, expression3)
```



- All expressions run **left → right**
- **Only the last expression is returned**

Quick Exam Trick

For:

```
i = (A, B, C)
```



Just remember:

```
i = C
```



But **A** and **B** still execute first.



casting operator

The **Casting Operator** in **C/C++** is used to **convert a value from one data type to another data type**. This process is called **type casting**.

◆ Syntax of Casting Operator

```
<> C++  
(type) value
```



Example:

```
<> C++  
(float) a
```



This converts `a` into a float.

◆ Example 1: Integer Division Without Casting

```
<> C++
```



```
#include <iostream>
using namespace std;

int main()
{
    int a = 5, b = 2;

    float result = a / b;

    cout << result;

    return 0;
}
```

Output

2



Explanation:

5 / 2 = 2



Because both are **integers**, C++ performs **integer division**.

◆ Example 2: Using Casting

```
</> C++

#include <iostream>
using namespace std;

int main()
{
    int a = 5, b = 2;

    float result = (float)a / b;

    cout << result;

    return 0;
}
```



Output

2.5



Explanation:

$$(\text{float})5 / 2 = 2.5$$


Now C++ performs **floating-point division**.

◆ Another Example

</> C++



```
#include <iostream>
using namespace std;

int main()
{
    float x = 9.8;

    int y = (int)x;

    cout << y;

    return 0;
}
```

Output

9



Explanation:

9.8 → converted to integer → 9



The **decimal part** is removed.



Summary

Cast	Meaning
<code>(float)a</code>	Convert <code>a</code> to float
<code>(int)x</code>	Convert <code>x</code> to integer
<code>(double)a</code>	Convert <code>a</code> to double

Simple Way to Remember

Casting operator means:

`(type) value`



Example:

`(int)3.7 → 3`
`(float)5 → 5.0`



 In exams, casting is usually used in questions like:

`(float)5/2`
`(int)7.9`



If you want, I can also ↪ show you the 4 types of casting in C++ (`static_cast` , `dynamic_cast` , `const_cast` , `reinterpret_cast`), but those usually come in advanced C++ topics.



`const_cast`

The `const_cast` operator in C++ is used to **add or remove the `const` qualifier** from a variable.

In simple terms, it lets you **change whether a variable is treated as constant or not**. 

◆ Syntax

</> C++



```
const_cast<type>(expression)
```

Example:

</> C++



```
const_cast<int*>(ptr)
```

◆ Why `const_cast` is Used

Sometimes a function expects a **non-const variable**, but you have a **const variable**.

`const_cast` allows you to **temporarily remove `const`** so the function can use it.

◆ Example

</> C++



```
#include <iostream>
using namespace std;

int main()
{
    const int x = 10;

    int *p = const_cast<int*>(&x);

    cout << *p;
```

```
    return 0;
}
```

Explanation

`x = 10` (constant) 

`const_cast` removes the `const` property:

```
int *p = const_cast<int*>(&x);
```



Now `p` is a pointer that can access `x` without `const` restriction.

Output:

10 

◆ Example with Modification

```
</> C++   
  
#include <iostream>  
using namespace std;  
  
int main()  
{  
    const int x = 10;  
  
    int *p = const_cast<int*>(&x);  
  
    *p = 20;  
  
    cout << *p;  
  
    return 0;  
}
```

Output:

20 

 Important: Modifying a truly **const variable like this** can cause undefined behavior, so it should be used carefully.

Key Points

Feature	Description
Operator	<code>const_cast</code>
Purpose	Add or remove <code>const</code>
Works with	Pointers and references
Use case	When a function requires non-const data

Simple Idea

`const_cast` → remove or add `const`



Example:

`const int x = 10`



Remove `const`:

`int* p = const_cast<int*>(&x)`



Quick Comparison of C++ Casts

Cast	Purpose
<code>static_cast</code>	Normal type conversion
<code>dynamic_cast</code>	Safe casting in inheritance
<code>const_cast</code>	Remove/add <code>const</code>

Cast**Purpose**`reinterpret_cast`

Low-level pointer conversion

If you'd like, I can also show ↪ a **very simple diagram explaining** `const`, **pointers, and** `const_cast` that makes it much easier to remember for exams.



dynamic cast

The `dynamic_cast` operator in C++ is used to **safely convert pointers or references within an inheritance hierarchy** (between base class and derived class). It checks at **runtime** whether the cast is valid. 🧠💻

◆ Syntax

</> C++

`dynamic_cast<new_type>(expression)`

Example:

</> C++

`dynamic_cast<Derived*>(basePointer)`

This tries to convert a **base class pointer** into a **derived class pointer** safely.

◆ Why `dynamic_cast` is Used

When working with **inheritance**, sometimes you have a **base class pointer** but want to access **derived class features**.

`dynamic_cast` checks at **runtime** if the conversion is safe.

If the cast **fails**:

- Pointer → returns **NULL**
- Reference → throws an **exception**

◆ Example

```
</> C++  
  
#include <iostream>  
using namespace std;  
  
class Base {  
public:  
    virtual void show() {}  
};  
  
class Derived : public Base {  
public:  
    void display() {  
        cout << "Derived class function";  
    }  
};  
  
int main()  
{  
    Base *b = new Derived;  
  
    Derived *d = dynamic_cast<Derived*>(b);  
  
    if (d != NULL)  
        d->display();  
  
    return 0;  
}
```

◆ Explanation

1 Base *b = new Derived;

- A base class pointer points to a **Derived object**.

2 `dynamic_cast<Derived*>(b);`

- C++ checks if `b` really points to a **Derived** object.

3 If correct:

`d → valid pointer`



Then:

`</> C++`



`d->display();`

Output:

Derived class function



◆ When Cast Fails

Example:

`</> C++`



```
Base *b = new Base;  
Derived *d = dynamic_cast<Derived*>(b);
```

Here the object is **Base**, not **Derived**, so:

`d = NULL`



Key Points

Feature	Description
Operator	<code>dynamic_cast</code>
Type	Runtime casting

Feature	Description
Works with	Pointers and references
Requirement	Classes must use virtual functions

 Simple Memory Trick

dynamic_cast → Safe casting in inheritance
(dynamic check at runtime)



 Quick Comparison

Cast	Purpose
static_cast	Compile-time conversion
dynamic_cast	Safe runtime casting
const_cast	Remove/add const
reinterpret_cast	Low-level pointer conversion

✅ If you'd like, I can also ↪ explain static_cast and reinterpret_cast with very simple examples, which will complete the 4 C++ casting operators (often asked in exams).

    

reinterpret cast

The `reinterpret_cast` operator in C++ is used for **low-level type conversion**. It tells the compiler to **treat a piece of memory as a different type without changing the actual data**. ⚙️

In simple words:

`reinterpret_cast` → reinterpret the memory as another type



◆ Syntax

⌕ C++



```
reinterpret_cast<new_type>(expression)
```

Example:

⌕ C++



```
reinterpret_cast<char*>(ptr)
```

◆ Basic Example

⌕ C++



```
#include <iostream>
using namespace std;

int main()
{
    int a = 65;

    char *p = reinterpret_cast<char*>(&a);

    cout << *p;

    return 0;
}
```

Explanation

```
a = 65
```



ASCII value 65 = 'A'

`reinterpret_cast` converts the **int pointer** into a **char pointer**.

So the program prints:

```
A
```



◆ Example with Pointers

```
</> C++
```



```
#include <iostream>
using namespace std;

int main()
{
    int a = 10;

    int *p = &a;

    long address = reinterpret_cast<long>(p);

    cout << address;

    return 0;
}
```

Explanation

Here:

```
int* → converted to long
```



So the **memory address of** `a` is stored as a number.

◆ Important Notes

Feature	Description
Type	Low-level casting
Safety	Not type-safe
Usage	Pointers, addresses, memory manipulation

Warning

`reinterpret_cast` is **dangerous** if used incorrectly because it **bypasses type safety**. It is mostly used in:

- System programming
- Memory manipulation
- Hardware-level programming

Complete C++ Cast Operators

Cast	Purpose
<code>static_cast</code>	Normal type conversion
<code>dynamic_cast</code>	Safe inheritance casting
<code>const_cast</code>	Add/remove const
<code>reinterpret_cast</code>	Low-level memory reinterpretation

Easy Way to Remember

```
static_cast    → normal conversion
dynamic_cast  → safe inheritance conversion
const_cast    → remove const
reinterpret_cast → treat memory as another type
```



✓ If you'd like, I can also show you ↪ a very simple visual diagram that explains all 4 C++ casts in 30 seconds, which makes them much easier to remember for exams.



static cast

The `static_cast` operator in C++ is used for **normal type conversions at compile time**. It converts one data type to another in a **safe and clear way**. 🔄

It is basically the **modern C++ replacement for the old C-style cast**.

◆ Syntax

```
</> C++  
  
static_cast<new_type>(expression)
```

Example:

```
</> C++  
  
static_cast<float>(a)
```

This converts `a` into a float.

◆ Example 1: Integer to Float

```
</> C++  
  
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int a = 5, b = 2;
```



```
float result = static_cast<float>(a) / b;  
  
cout << result;  
  
return 0;  
}
```

Explanation

Without casting:

5 / 2 = 2



With `static_cast`:

5 / 2 = 2.5



Output:

2.5



◆ Example 2: Float to Integer

```
</> C++  
  
#include <iostream>  
using namespace std;  
  
int main()  
{  
    float x = 9.8;  
  
    int y = static_cast<int>(x);  
  
    cout << y;  
  
    return 0;  
}
```



Result



The decimal part is removed.

◆ Example 3: Base and Derived Class

</> C++



```
#include <iostream>
using namespace std;

class Base {};
class Derived : public Base {};

int main()
{
    Base *b = new Derived;

    Derived *d = static_cast<Derived*>(b);

    return 0;
}
```

Here the pointer **b** is converted into **Derived** pointer.



Key Features

Feature	Description
Type	Compile-time casting
Safety	Safer than C-style cast
Uses	Numeric conversion, pointer conversion



Easy Way to Remember

`static_cast` → normal type conversion

Example:

```
static_cast<int>(3.7) → 3  
static_cast<float>(5) → 5.0
```



⚡ Final Comparison of C++ Casts

Cast	Purpose
<code>static_cast</code>	Normal conversions
<code>dynamic_cast</code>	Safe runtime casting (inheritance)
<code>const_cast</code>	Remove/add const
<code>reinterpret_cast</code>	Low-level memory casting

✅ Quick exam tip

If the question asks “convert int to float or float to int in modern C++”, the best answer is:

`static_cast`

operator precedence in c++

Operator precedence in C++ determines **which operator is evaluated first** in an **expression** when multiple operators appear together. 🧠💻

If two operators have the same precedence, **associativity** decides the evaluation order.

Example:

</> C++

Multiplication `*` has **higher precedence** than `+`, so:

5 * 2 = 10
10 + 10 = 20

Result = 20

 Operator Precedence Table (High → Low)

Precedence	Operators	Description
1 (Highest)	<code>() [] -> .</code>	Parentheses, array, member access
2	<code>++ --</code>	Increment / Decrement
3	<code>* / %</code>	Multiplication, Division, Modulus
4	<code>+ -</code>	Addition, Subtraction
5	<code><< >></code>	Bitwise shift
6	<code>< <= > >=</code>	Relational
7	<code>== !=</code>	Equality
8	<code>&</code>	Bitwise AND
9	<code>^</code>	Bitwise XOR
10	<code>,</code>	<code>,</code>
11	<code>&&</code>	Logical AND
12	<code>,</code>	

Precedence	Operators	Description
13	<code>?:</code>	Conditional (ternary)
14	<code>` = += -= *= /= %= <<= >>=</code> <code>&= ^=</code>	<code>=`</code>
15 (Lowest)	<code>,</code>	Comma operator

◆ Example 1

`</> C++`

Step:

`3 * 2 = 6`
`5 + 6 = 11`

Output:

`11`

◆ Example 2

`</> C++`

Parentheses have **highest priority**.

`5 + 3 = 8`
`8 * 2 = 16`

Output:

16



◆ Example 3

</> C++



+ and **-** have same precedence, so evaluation is **left → right**.

```
10 - 4 = 6
6 + 2 = 8
```



Output:

8



🧠 Important Rules

- 1 Parentheses **()** always execute first
- 2 Multiplication/Division before Addition/Subtraction
- 3 Logical operators are evaluated after relational operators
- 4 Assignment happens near the end

⚡ Quick Memory Trick

Remember this order:

() → ++ -- → * / % → + - → << >> → < > → == → & → ^ → | → && → || → ?: → =



✅ If you'd like, I can also show you ↪ 5 tricky operator precedence questions that professors love to put in C++ exams (they look confusing but are easy once you know